

Transformer

↓
Encoder-only models ✓

↓
Encoder-Decoder models ✓

↓
Decoder-only models ✓

↓
Early model training pipeline (BERT)

↓
Modern LLM families

↓
Modern LLM training pipeline

↓
SLM concepts ← (FT)

↓
Multimodal concepts:

↓
Multimodal model families

↓
Leaderboards ✓

↓
APIs to access models

GPT MODERN LLM

CHATGPT

Quick Recap

Langchain

(2017-18)

MODEL

EARLY STAGE LLM

✓ (1) BERT

✓ (2) GPT

✓ (3) BART

✓ (4) TS

MLM

NTP(AR)

(2017)

ULMFIT

(1) Pretraining

(Self-Supervised)

(2) SFT

→ instruction FT

output head

CE Embedding

⌈ ⌋ ⌈ ⌋ ⌈ ⌋ ⌈ ⌋ ⌈ ⌋

⌈ ⌋ ⌈ ⌋ ⌈ ⌋ ⌈ ⌋ ⌈ ⌋

↑ ↑ ↑ ↑

< > < > < >



The heart of Transformer is self-attention, where the model understands each token in a sentence in the context of all the other tokens.

✓ Encoder-Only Models:

BERT ✓ ←

RoBERTa ✓

DistilBERT ✓

DeBERTa ✓

Use of these models:

Text classification: Sentiment analysis, Named Entity Recognition, POS

Embedding generation ✓

Input: "I hate this product."

Output: Negative

Input: "Virat Kohli lives in India."

Output: Virat Kohli = Person, India = Location

Input: "Ram eats mango."

Output: Ram = Noun, eats = Verb, mango = Noun

Encoder-only models are best for deeply understanding text, but they are not naturally designed to generate text.

Google | Meta | Apple

PAD | RS | AI

AI
Dev

90-95-99.1.

Encoder-Decoder Models

T5 ✓ → Google
BART ✓ → Meta
mT5 ✓ → Google
FLAN-T5 ✓ → Google

2019-2020

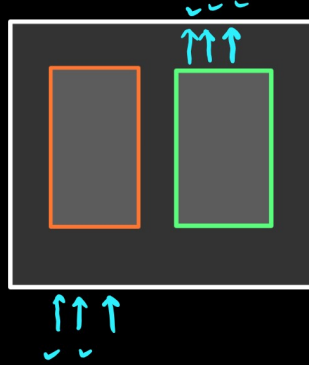
Use case:

Translation

Summarization

Question answering

T5/BART



Input: "How are you?"

Output: "आप कैसे हैं?"

Input: "Transformers use self-attention to understand relationships between words and are used in modern LLMs."

Output: "Transformers use self-attention and power modern LLMs."

Context: "BERT is an encoder-only model."

Question: "What type of model is BERT?"

Output: "Encoder-only model."

T5, BART

Encoder-decoder models are best when we need to understand the input and generate a new structured output, such as translation or summarization.

Decoder-Only Models:

GPT ✓
LLaMA ✓
Mistral ✓
Deepseek ✓

Use case: text generation

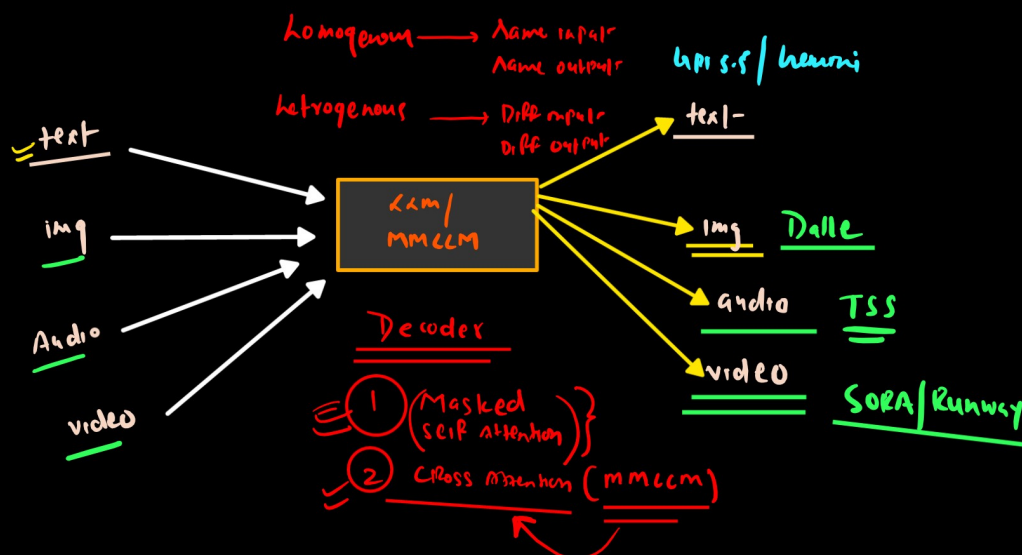
Decoder-only models are general-purpose generative models.

They predict the next token one by one and can generate many types of outputs.

It can generate any thing

- ✓ Conversational AI: Chatbots, QA assistants
- Code generation: Python, JavaScript, SQL, APIs
- Content writing: Blogs, emails, scripts, summaries
- Reasoning: Step-by-step problem solving

Today's most popular LLMs are based on the decoder-only style because their core task is next-token prediction and generation.



Early Stage Model Training

Self-supervised / unsupervised (pretraining)

Data → non-label data

Label is being created

Supervised fine-tuning

(BERT)

Text classification

Stage 1: Self-Supervised Pretraining

In this stage, the model learns language patterns from a large amount of unlabeled text data.

Example: BERT

BERT uses Masked Language Modeling.

(MLM)

Pretraining

Language Pattern

Input: The cat [MASK] on the mat.

Target: sat

Here, the model learns to predict the missing word using both left and right context.

Example: GPT

GPT uses Next Token Prediction.

Input: I love machine

Target: learning

Here, the model looks at previous tokens and predicts the next token.

Another example:

Input: Artificial intelligence is

Target: powerful

Then the sequence becomes:

Artificial intelligence is powerful

Now GPT predicts the next token again.

Input: Artificial intelligence is powerful

Target: because

So GPT learns by repeatedly predicting the next token from huge text data.

In pretraining, the model learns language structure, grammar, facts, and patterns without manually labeled data.

Stage 2: Supervised Fine-Tuning

In this stage, the model is trained on labeled data for a specific task.

Example:

Input: This movie is amazing.

Label: Positive

Another example:

Input: Translate "How are you?" into Hindi.

Output: आप कैसे हैं?

Teaching Line

Supervised fine-tuning converts a general pretrained model into a task-specific model using labeled examples.

Fine-tuning turns a general language model into a task-specialist model.

Modern LLM Training Pipeline

Correct modern pipeline:

Large-scale pretraining

↓

Supervised Fine-Tuning / Instruction Tuning

↓

Preference Alignment

Stage 1: Pretraining

The model is trained on internet-scale or curated data.

Goal:

Next token prediction

Learns:

Language Facts Reasoning patterns Code World knowledge Basic instruction patterns

Stage 2: Supervised Fine-Tuning / Instruction Tuning

The model is trained on human-written instruction-response pairs.

Example:

User: Explain Transformer in simple words. Assistant: Transformer is a neural network architecture...

Learns:

Instruction following Answer formatting Helpful response style Task completion

Stage 3: Preference Alignment

The model is aligned using human or AI preference data.

Methods:

RLHF DPO ORPO GRPO Constitutional AIRLAI

Learns:

Helpful answer

Safe answer

Polite tone

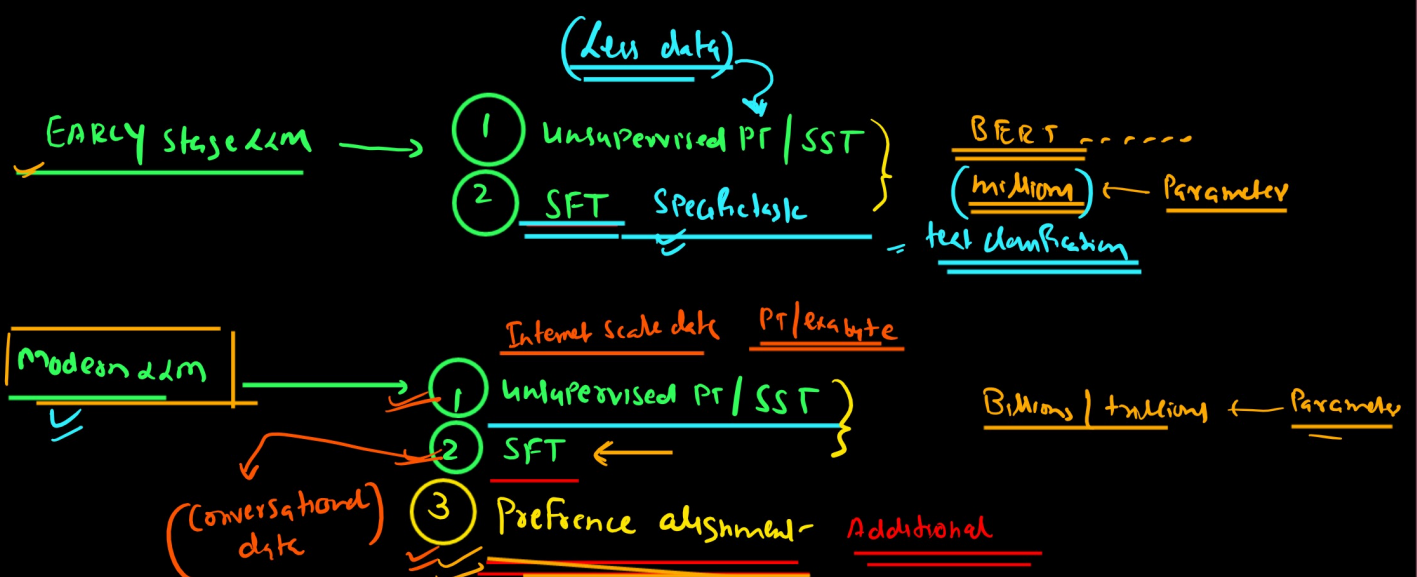
Better reasoning

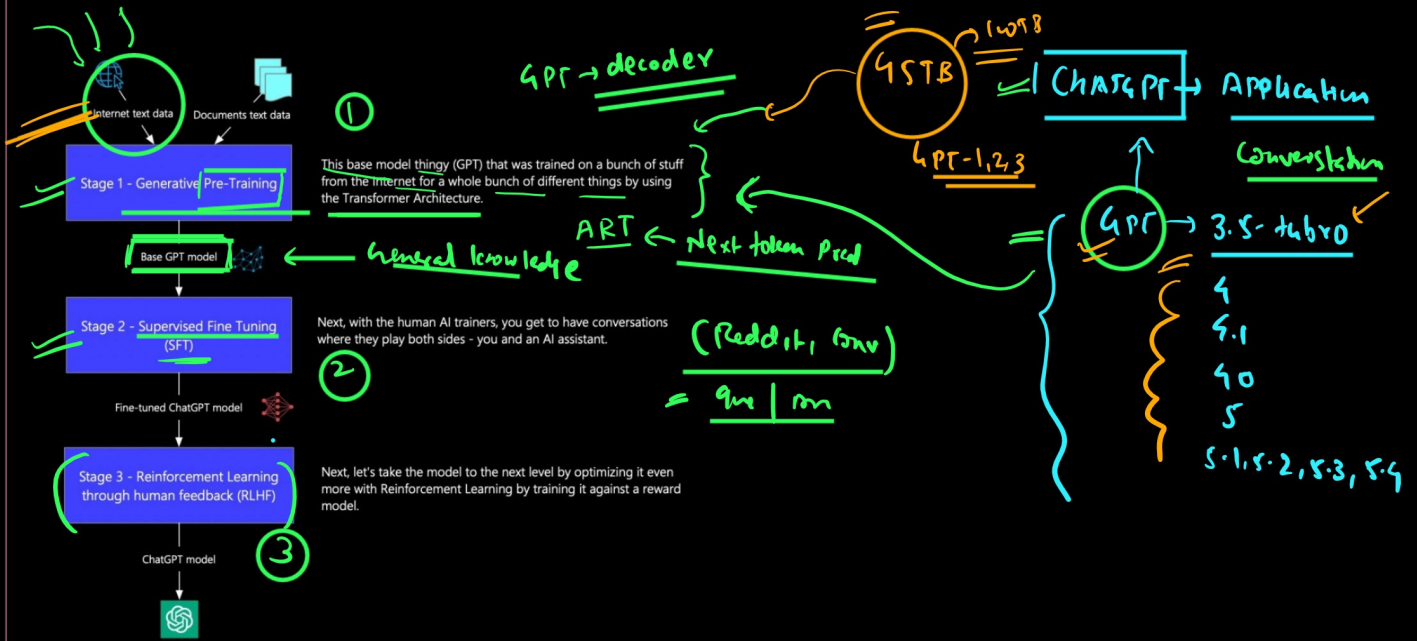
Less harmful output

Better refusal behavior

Teaching Line

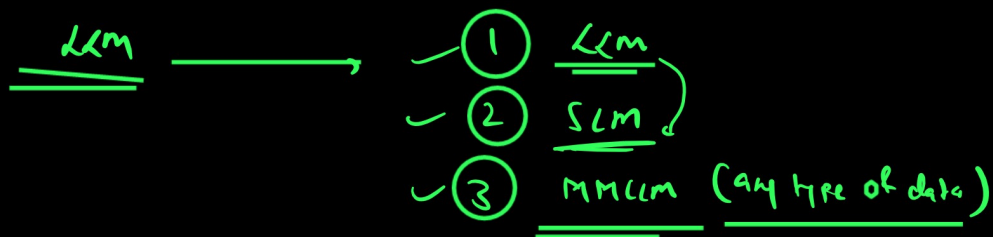
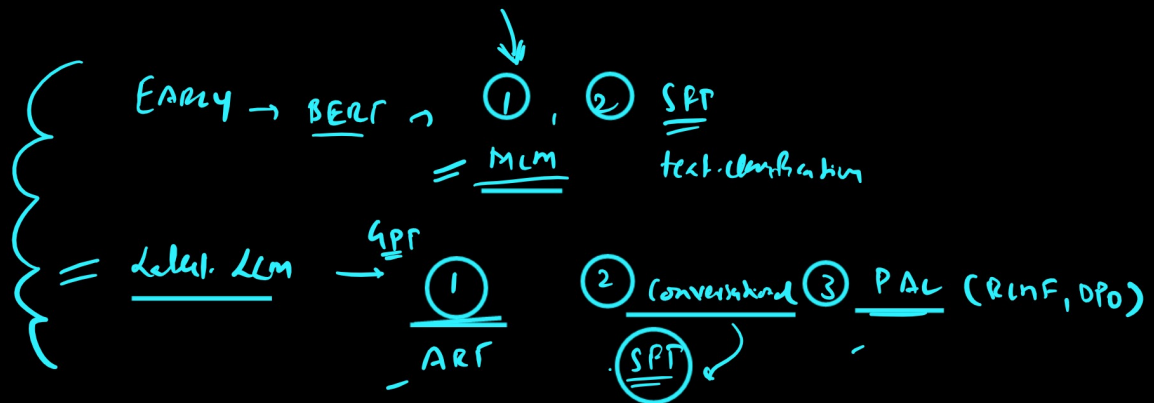
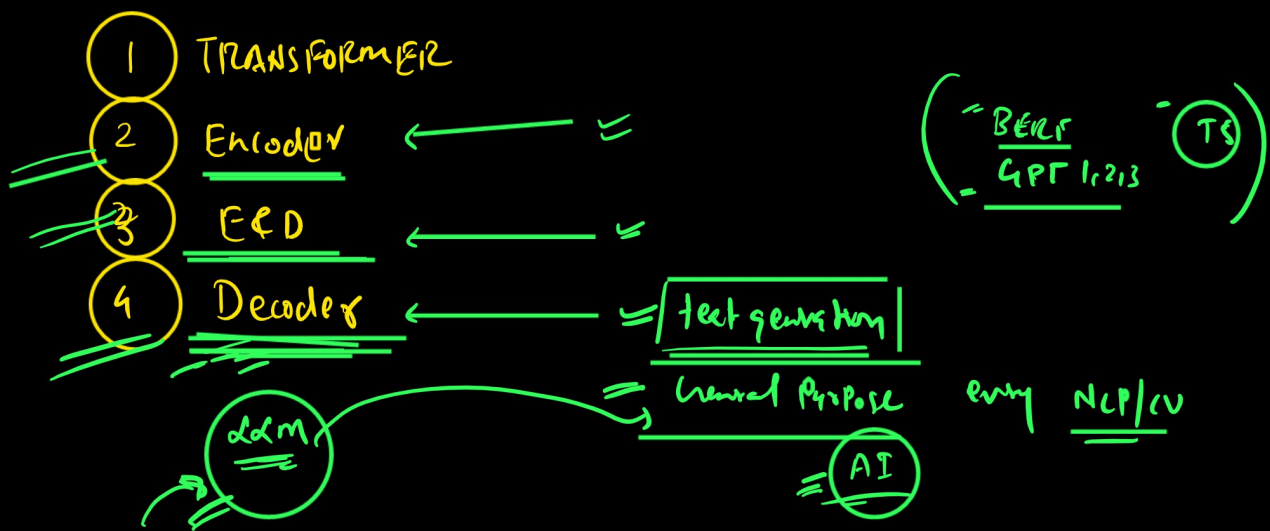
Pretraining gives the model knowledge, SFT teaches the model instruction following, and preference alignment makes the model behave like a human-like assistant.

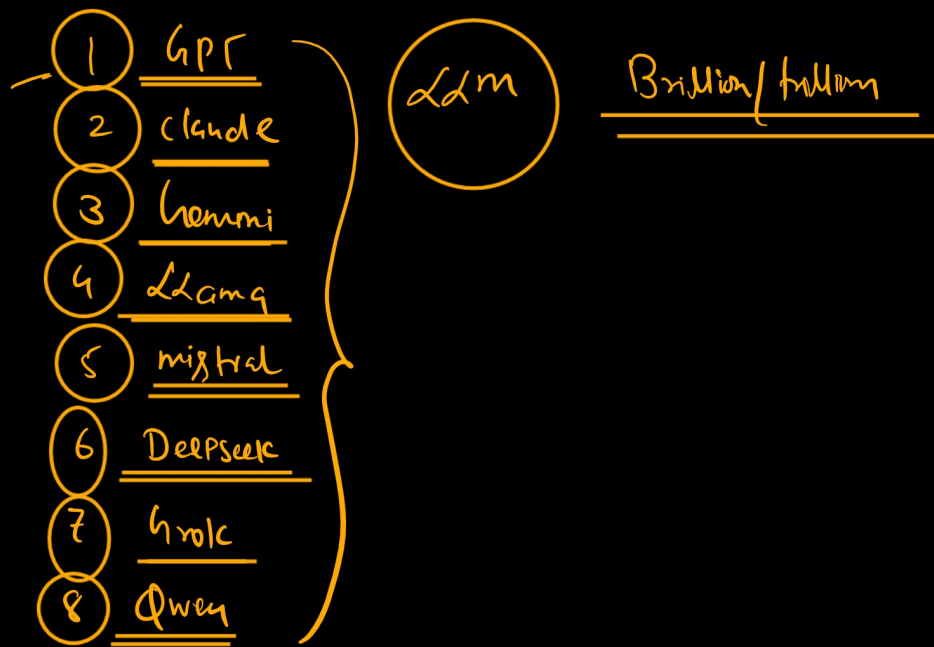




Question	Answer 1	Answer 2	Preference
How are you feeling today?	I am feeling happy	I am feeling intense	1
What did you think of the movie?	It was absolutely wonderful	It was surprisingly intense	1
How was your day at work?	I finished all my tasks	I struggled with deadlines	1
How do you view this project?	This project excites me	I have no opinion	1
Did you enjoy the meal?	The meal was bland	The meal was fantastic	2
How do you feel about traveling?	I love exploring new places	Traveling makes me nervous	1
What do you think of your team?	My team is very supportive	My team is sometimes difficult	1
How was your presentation?	I was nervous but confident	I was totally unprepared	1
Did you like the book?	The book was fascinating	The book was confusing	1
How did the interview go?	I think I made a good impression	I'm unsure how I did	1







model > job ← LLM

SLM Concepts

SLM = Small Language Model.

The goal of an SLM is:

Fewer parameters, lower cost, faster inference, and strong performance on specific tasks.

Less Parameters

Model < 10B

SLM

9B, 8B, 7B

Phi family

Gemma small variants

Qwen small variants

LLaMA small variants

Mistral 7B style models

Microsoft

Google

10B

SLM

SLM use cases:

On-device AI

Enterprise internal chatbot

Domain-specific RAG

Low-cost inference

Fast customer support bot

Private deployment

Edge AI

SLM

SARVAM AP

SLM

Date

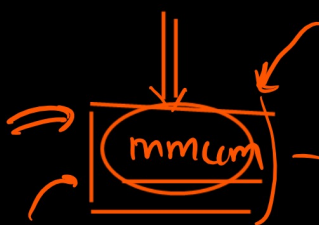
PT

12/13

Not every problem needs a GPT-level giant model. In many business use cases, a well-tuned SLM is cheaper, faster, and more controllable.

LAM/SLM

diff



GPT 5.5, Claude 4.2, Gemini 3

Audio
trans
text

text to text
text to image
img to text
img to img

text

LAM

text

Image

Stable diffusion

Images

Diffusion

Process

Denosing

Unet

transformer based denoiser

Vit, CLIP

Vision transformer

Image

Multimodal Concept: ViT

ViT = Vision Transformer.

Concept:

Image → patches → patch embeddings → Transformer encoder → image representation

Example:

224x224 image

↓

16x16 patches

↓

Patch tokens

↓

Transformer

ViT treated images as patches, similar to words, and made Transformers powerful in computer vision.

The ViT paper showed that a pure Transformer can be applied directly to image patches and perform strongly on image recognition when trained at scale.

CLIP = Contrastive Language-Image Pretraining.

Concept:

Image encoder + Text encoder

Training:

Correct image-text pair ko close lao

Wrong image-text pair ko far karo

Example:

Image: dog running on grass

Text: "a dog running on grass"

CLIP image aur text ko same embedding space me map karta hai.

Use cases:

Image search

Zero-shot classification

Image-text matching

Text-guided image retrieval

Multimodal retrieval

CLIP brought images and text into a shared semantic space. It became a foundational block of multimodal AI.

ViT

→ Converts an image into patches/tokens

CLIP

→ Brings image and text into the same semantic space

BLIP / BLIP-2

→ Image understanding + captioning + VQA

Diffusion

→ Generating an image from noise

VAE

→ Takes an image into a compressed latent space

U-Net / DiT (Diffusion Transformer)

→ Denoising engine

Cross-Attention

→ Guides image generation using the text prompt

Diffusion = generation process

Transformer = architecture

Random noise → step-by-step denoise → final image/video/audio

Stable Diffusion 1.x

Stable Diffusion 2.x

SDXL

In these, the main denoising network is mostly:

U-Net

Flow:

Text prompt

↓

Text encoder, often CLIP/T5

↓

U-Net denoiser

↓

VAE decoder

↓

Final image

Prompt: "a dog wearing sunglasses"

↓

CLIP Text Encoder

↓

Text embedding

↓

Diffusion U-Net / Transformer denoiser

↓

Generated image

ViT ≠ Diffusion

CLIP ≠ Diffusion

DiT = Diffusion process + Transformer architecture

A diffusion model is not a Transformer by default.

Older diffusion models mostly used U-Net.

Modern diffusion models use Transformer-based denoisers, which are called DiT.

Different Multimodal Model Names

Image-to-Text / Vision-Language Understanding

BLIP
BLIP-2
LLaVA
Flamingo
Kosmos
GPT-4V / GPT-4o style
Gemini Vision style
Qwen-VL

Text-to-Image Models

DALL-E
Stable Diffusion
SDXL
Imagen
Midjourney
FLUX
Firefly

Audio-to-Text Models

Whisper
Wav2Vec 2.0
HuBERT
DeepSpeech
Conformer

Text-to-Audio / Text-to-Speech Models

Tacotron 2
FastSpeech
VITS
Bark
AudioLM
MusicLM
ElevenLabs-style systems

Text-to-Video Models

Sora

Veo

Kling

Runway Gen series

Pika

Luma Dream Machine

Stable Video Diffusion

Video-to-Text / Video Understanding Models

VideoBERT

VideoCLIP

Video-LLaVA

VideoChatGPT

Gemini multimodal

GPT-4o style systems

Image-to-image models take an existing image as input and generate a modified image based on a prompt, style, mask, or control signal.

Input image + instruction/prompt



Modified / enhanced / transformed image

Image-to-Image Use Cases

Image editing

Style transfer

Inpainting

Outpainting

Background change

Sketch-to-image

Low-quality to high-quality image

Colorization

Pose/control-based generation

Object removal/addition

Popular Image-to-Image Models

Stable Diffusion img2img

ControlNet

InstructPix2Pix

SDEdit

DALL·E Editing

Photoshop Generative Fill

FLUX Kontext

IP-Adapter

InstantID

PhotoMaker

Input image: A normal dog photo

Prompt: Convert this dog into a cartoon style

Output: Cartoon-style dog image

Audio-to-audio: Audio-to-audio models take an audio input and generate a modified audio output, such as translated speech, converted voice, cleaned speech, or separated music.

Input audio



Modified / converted / enhanced audio

Audio-to-Audio Use Cases

Voice conversion
Speech-to-speech translation
Voice cloning
Noise removal
Audio enhancement
Music separation
Speaker style transfer
Audio super-resolution
Accent conversion
Dubbing

Popular Audio-to-Audio Model/System Examples

SeamlessM4T
AudioPaLM
RVC
Demucs
VoiceFixer
SpeechT5
OpenVoice
FreeVC
so-vits-svc

Simple Examples

1. Voice Conversion

Input: Your voice saying "Hello"

Output: Same sentence in another person's voice style

2. Speech-to-Speech Translation

Input: Hindi speech

Output: English speech

3. Noise Removal

Input: Noisy meeting recording

Output: Clean speech audio

4. Music Separation

Input: Full song

Output: Vocals + drums + bass + instruments separated

Video-to-video : Video-to-video models take an existing video as input and generate a modified video based on a prompt, style, mask, motion, or control signal.

Input video



Modified / transformed / enhanced video

Video-to-Video Use Cases

Video editing

Video style transfer

Background change

Object removal/addition

Face animation

Motion transfer

Video enhancement

Low-quality to high-quality video

Frame interpolation

Color grading

Text-guided video editing

Popular Video-to-Video Models / Systems

Runway Gen series

Pika

Luma Dream Machine

Sora-style video editing

Stable Video Diffusion

AnimateDiff

VideoCrafter

TokenFlow

Tune-A-Video

Rerender-A-Video

EbSynth

FILM / RIFE

Simple Examples

1. Video Style Transfer

Input: Normal walking video

Prompt: Convert it into anime style

Output: Same motion, anime-style video

2. Background Change

Input: Person talking in a room

Prompt: Change background to a studio setup

Output: Same person, studio background

3. Object Removal

Input: Road video with unwanted car

Prompt: Remove the car

Output: Clean road video

4. Video Enhancement

Input: Low-quality old video

Output: High-resolution enhanced video

Text/LLM APIs

OpenAI API
Anthropic API
Google Gemini API
Google Vertex AI
AWS Bedrock
Azure Cloud Foundry
Together AI
Groq
Open Router
Fireworks AI
Hugging Face Inference Providers
Replicate

Image APIs

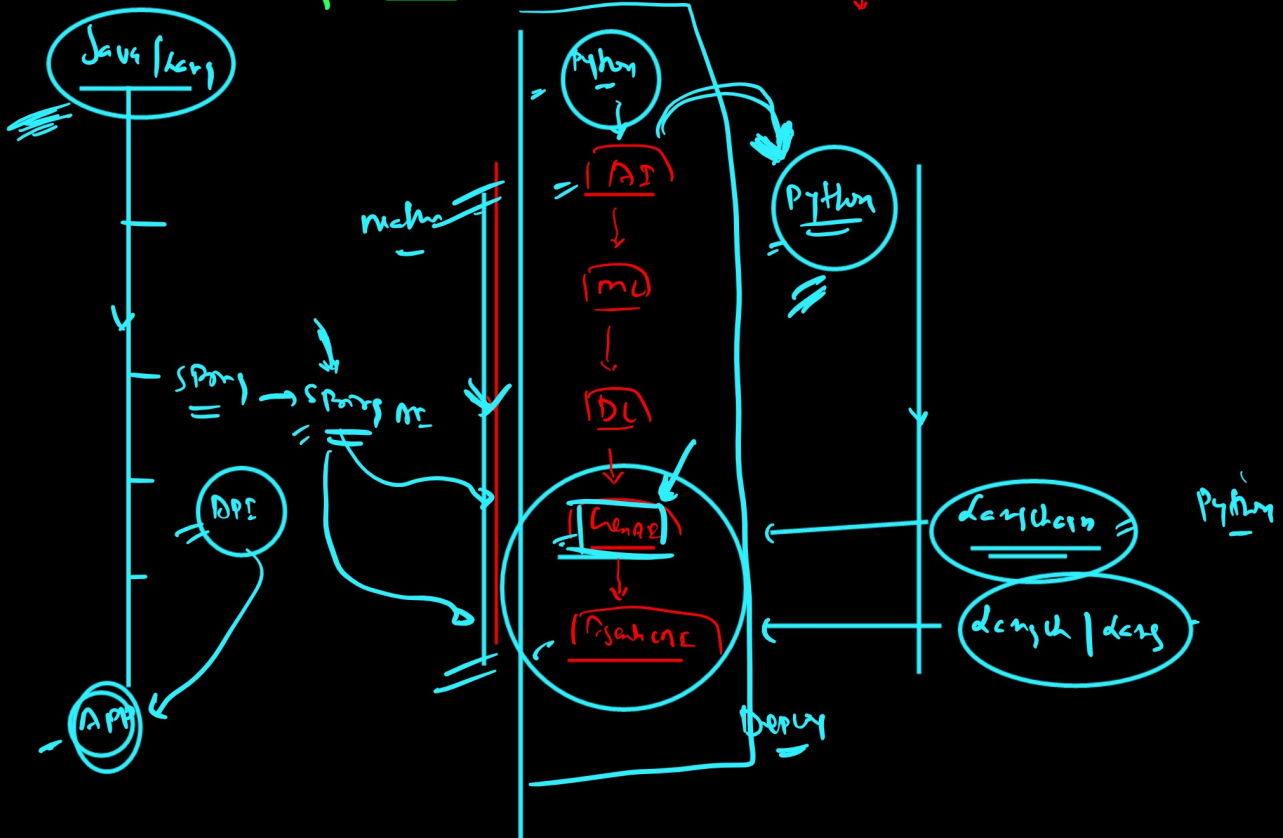
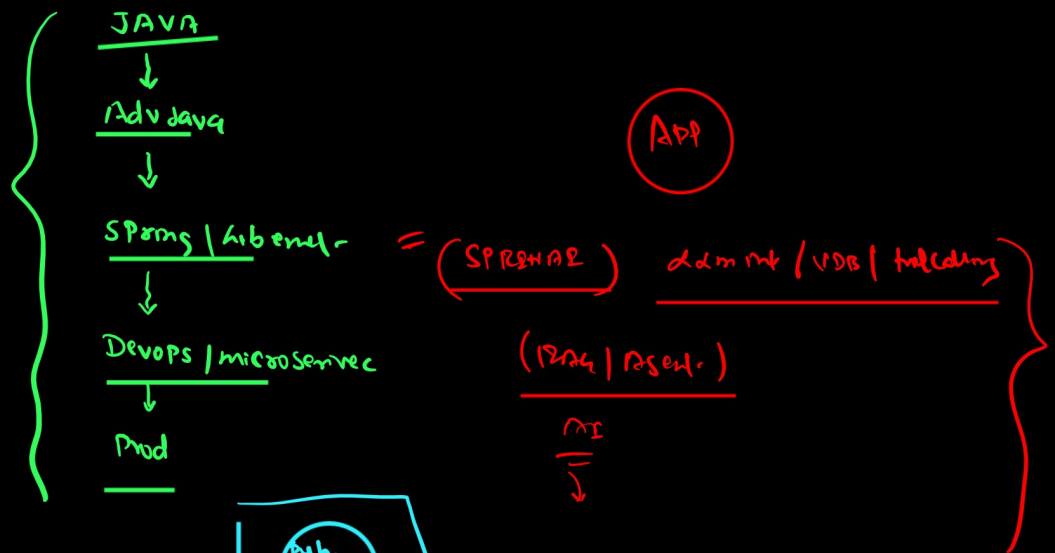
OpenAI
Stability AI
Replicate
Hugging Face
Fal.ai
Runware
Together
Fireworks

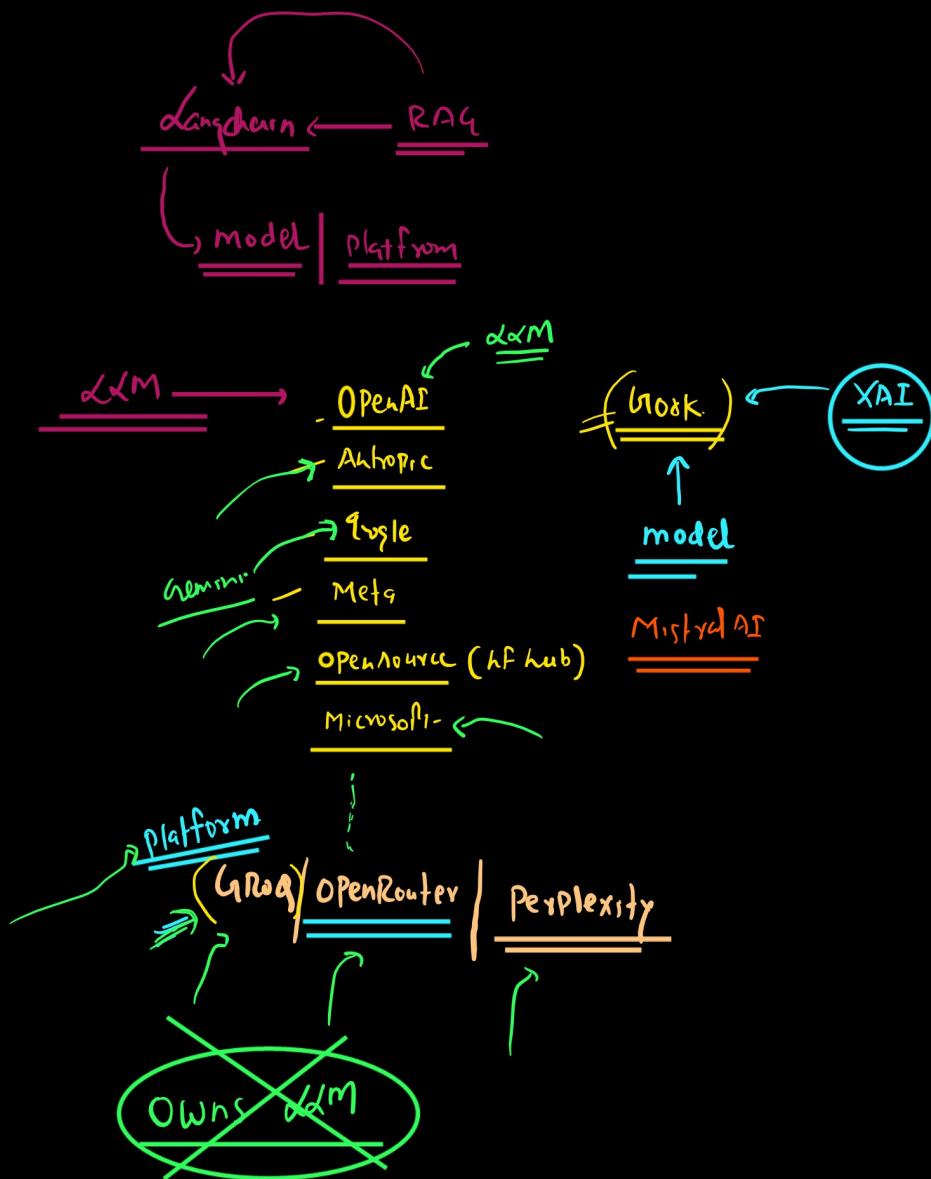
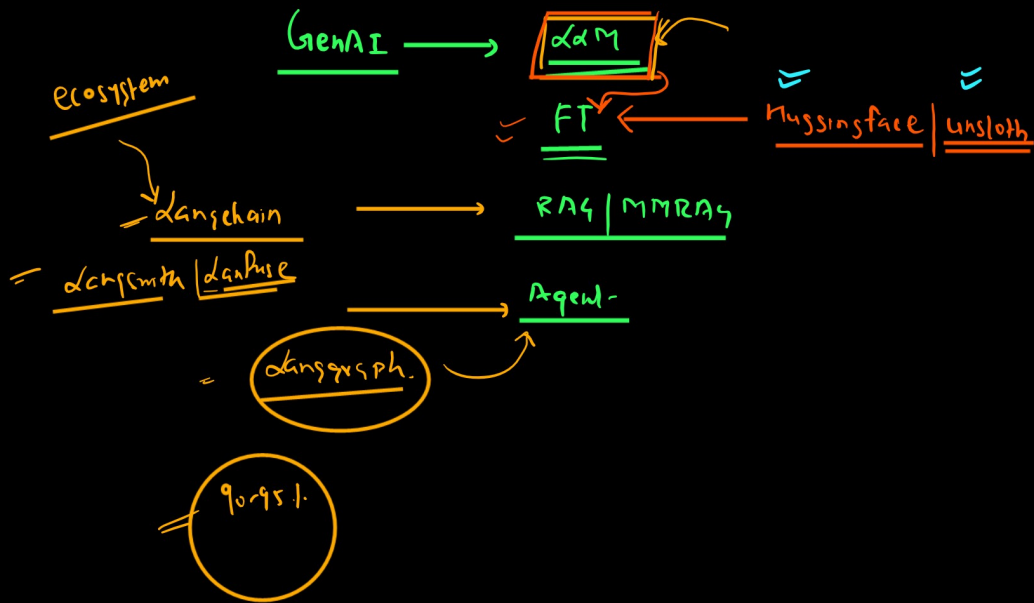
Audio APIs

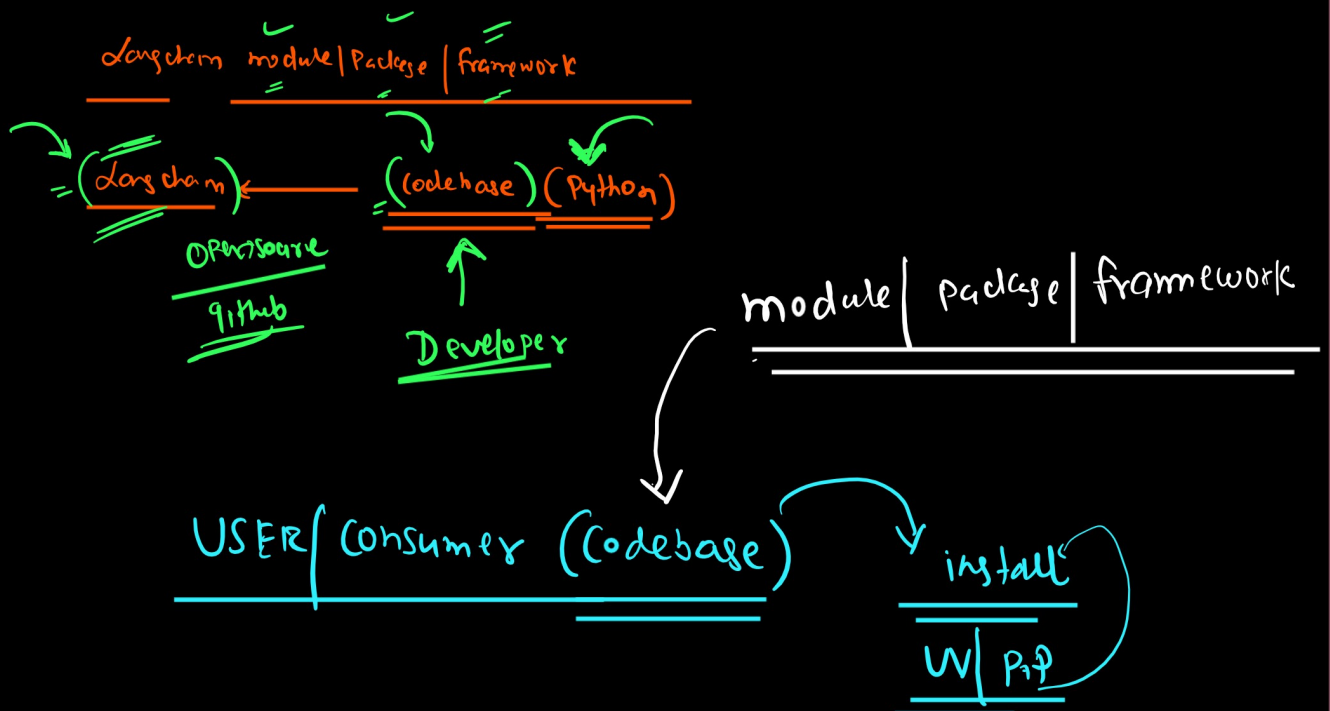
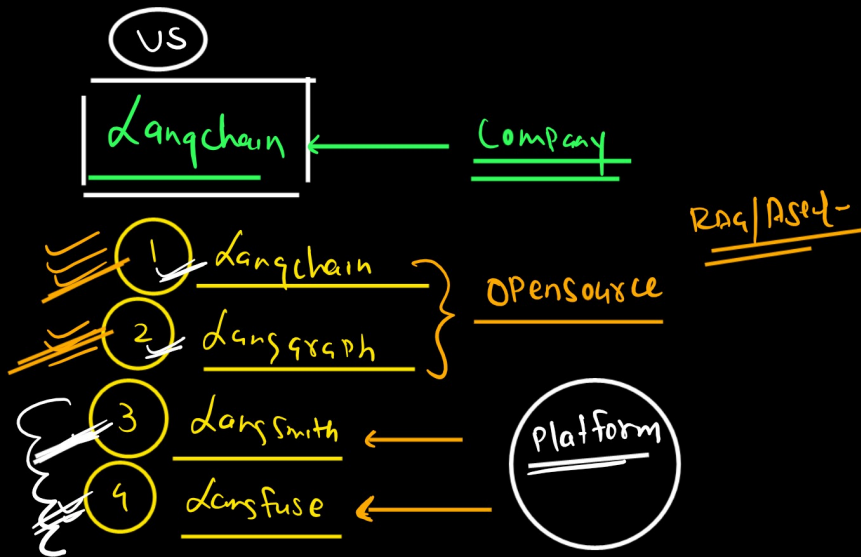
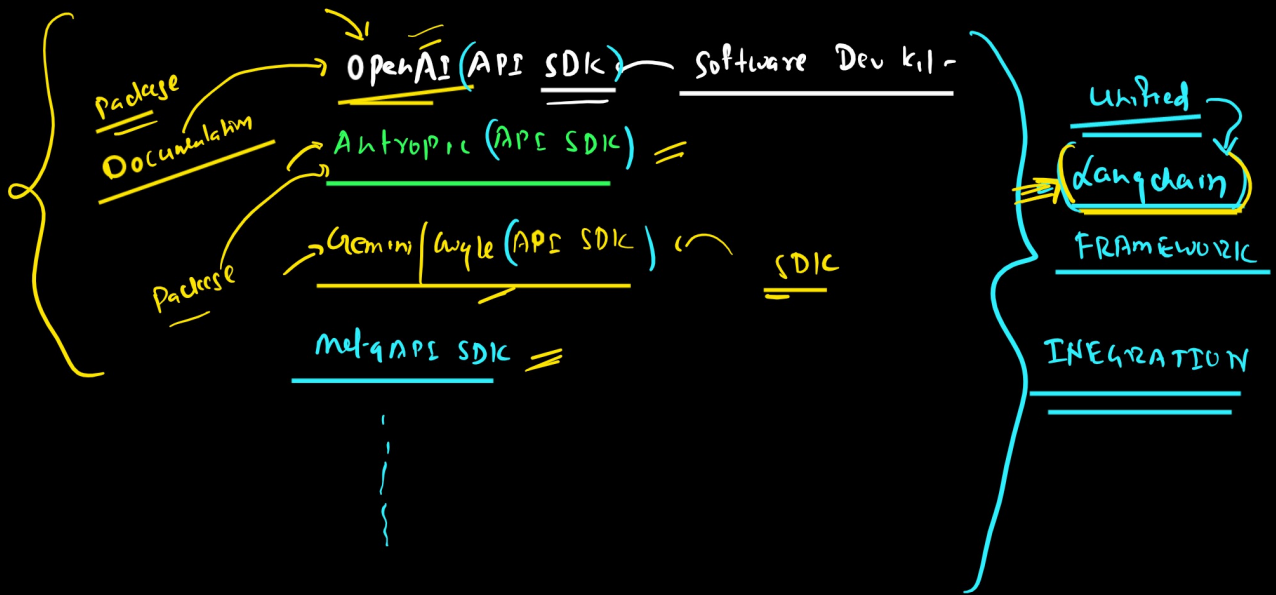
OpenAI
ElevenLabs
Deepgram
AssemblyAI
Google Speech
Azure Speech
Nova
ServemAI

Video APIs

Runway
Pika
Luma
Replicate
Fal.ai
Google Veo access paths
OpenAI Sora access paths







OpenAI has its own SDK. Groq has its own SDK. Anthropic has its own SDK. Google has its own SDK. So technically, yes, we can directly call models using their official SDKs.

SDK is mainly for calling one provider's model. LangChain is for building complete LLM applications around models.

OpenAI SDK = OpenAI engine

Groq SDK = Groq engine

Anthropic SDK = Anthropic engine

Google SDK = Google engine

But when we build real-world GenAI applications, we do not only need an engine.



```
from openai import OpenAI
```

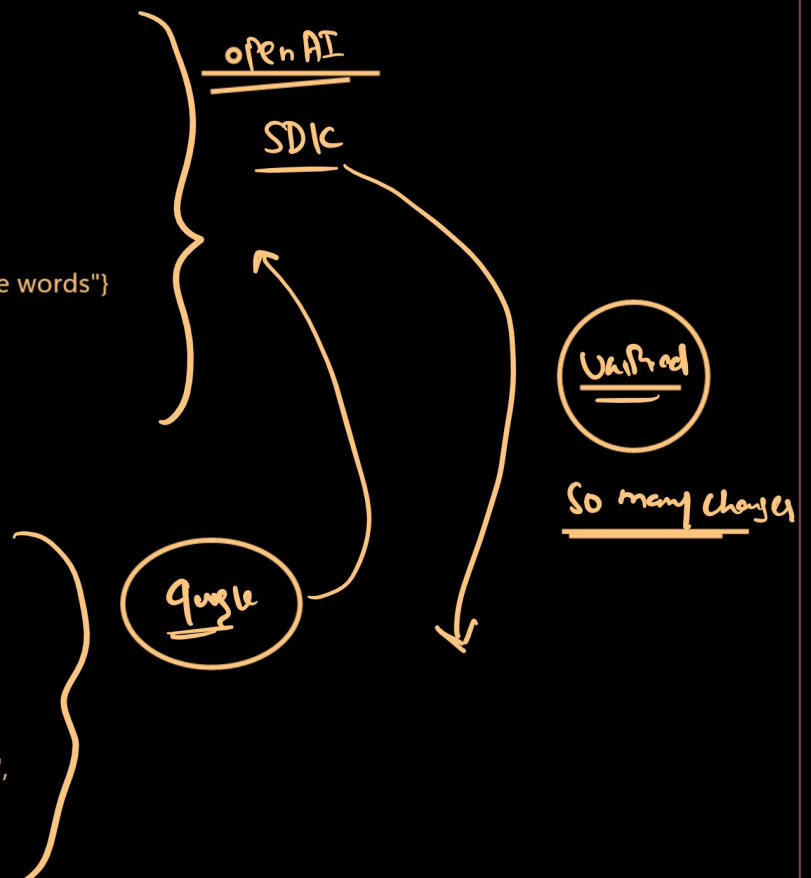
```
client = OpenAI()
```

```
response = client.chat.completions.create(  
    model="gpt-4o-mini",  
    messages=[  
        {"role": "user", "content": "Explain RAG in simple words"}  
    ]  
)  
print(response.choices[0].message.content)
```

```
from google import genai
```

```
client = genai.Client()
```

```
response = client.models.generate_content(  
    model="gemini-3-flash-preview",  
    contents="Explain how AI works in a few words",  
)  
print(response.text)
```



```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model="gpt-4o-mini")
```

```
response = llm.invoke("Explain RAG in simple words")
```

```
print(response.content)
```

```
from langchain_groq import ChatGroq
```

```
llm = ChatGroq(model="llama-3.1-8b-instant")
```

```
response = llm.invoke("Explain RAG in simple words")
```

```
print(response.content)
```

```
from langchain_google_genai import ChatGoogleGenerativeAI
```

```
model = ChatGoogleGenerativeAI(
```

```
model="gemini-2.0-flash"
```

```
)
```

```
ai_msg = model.invoke(messages)
```

```
ai_msg
```

UniReel

Why not specific SDIC

Why Langchain integration

Prompt(Input message) ---->LLM----->output(output message or AI-Message)

user_message

or

system_message

user_message

we have two way of defining ths system and user message

1. first simple tuple format(older way of passing message but still relevant)

```
messages = [  
    ("system", "You are a helpful assistant that translates English to Hindi. Translate the user sentence."),  
    ("human", "I love programming.")  
]
```

2. second using SystemMessage class and HumanMessage class(latest way of defining message)

latest LangChain variant

```
messages = [  
    SystemMessage(content="You're a helpful assistant"),  
    HumanMessage( content="What happens when an unstoppable force meets an immovable object?"),  
]
```

SystemMessage or system -->we are defining the permanent behviour of llm in current active session

HumanMessage or human --> whatever human wanted to provide input it is that message

Openrouter

openrouter providing you the models

you can generate api key and access the model

LangChain

it is not providing you the model

it is just giving you the integration

menans generate the api key from the respective platform(openai, claude, gemini or openrouter) and access the model

RAG, Agent

Prompt templates

Memory Managment

Document loaders

Text splitters

Embeddings

Vector databases Integration

Retrievers

Tools

Agents module

Tracing

Evaluation

Model switching

From Next Class -5 classes

Finetuning and Huggingface(in a detail) and Unsloth

Project: LLM OpenAI/Claude

Openrouter
huggingface
groq
Ollama